Floor Map 抽取算法详细规范

> 版本：v1.0
>
> 日期：2026-04-30
>
> 关联：FLOOR_MAP_HYBRID_RENDERING_PLAN.md §4
>
> 目标读者：实现 `scripts/floor_map/*.py` 的开发者 / Copilot 子代理

---

0. 总体约束

- 语言：Python 3.11+，依赖 `ezdxf>=1.4`，无新增依赖
- 位置：`scripts/floor_map/`（与 `split_dxf_by_floor.py` 同级）
- 纯函数：所有 detector 不得修改 DXF 文档，不得读写文件，只接收实体列表 + 坐标映射器，返回 dataclass 列表
- 失败策略：单个实体抛异常时记录 warning 并跳过，不影响其他实体；所有 detector 必须在最坏情况返回 `[]` 而非抛错
- 坐标系：所有 detector 接收 DXF 原始 mm 坐标的实体；通过 `coordinate_mapper.dxf_to_viewport(x, y) → (sx, sy)` 转换后再写入 `floor_map.json`

---

1. 数据结构（`scripts/floor_map/__init__.py`）

```
from dataclasses import dataclass, field
from typing import Literal, Optional, Tuple, List

Side = Literal["N", "S", "E", "W"]
SourceTag = Literal["auto", "manual"]
StructureType = Literal["core", "elevator", "stair", "restroom", "equipment", "corridor"]

@dataclass
class Rect:
    x: int   # SVG viewport 像素坐标
    y: int
    w: int
    h: int

@dataclass
class Outline:
    type: Literal["rect", "polygon"]
    rect: Optional[Rect] = None
    points: Optional[List[Tuple[int, int]]] = None  # type=polygon 时使用

@dataclass
class Structure:
    type: StructureType
    rect: Rect
```

```
    label: Optional[str] = None          # 例：「核心筒」「消防楼梯」
    code: Optional[str] = None           # 例：电梯编号 E1/E2
    gender: Optional[Literal["M", "F", "unknown"]] = None  # 仅 restroom
    source: SourceTag = "auto"
    confidence: float = 1.0              # 自动识别置信度，0.0-1.0

@dataclass
class Column:
    type: Literal["column"]
    point: Tuple[int, int]               # SVG 中心点
    source: SourceTag = "auto"

@dataclass
class Window:
    side: Side
    offset: int                          # 沿外轮廓某条边的位置（像素）
    width: int                           # 窗洞宽度（像素）

@dataclass
class NorthArrow:
    x: int
    y: int
    rotation_deg: float = 0.0
```

---

2. `coordinate_mapper.py`

2.1 公共 API

```
def build(
    dxf_bbox: ezdxf.math.BoundingBox,
    viewport: Tuple[int, int] = (1200, 800),
    padding: int = 20,
) -> CoordinateMapper: ...

class CoordinateMapper:
    def dxf_to_viewport(self, x: float, y: float) -> Tuple[int, int]: ...
    def dxf_length_to_pixel(self, length: float) -> int: ...
    @property
    def scale(self) -> float: ...                # 单位：viewport_px / dxf_mm
```

2.2 算法

1. 计算 DXF bbox 的宽 `dw` 和高 `dh` （mm）
2. 计算可用 viewport 区域： `(W - 2*padding, H - 2*padding)`
3. 必须等比例缩放： `scale = min((W-2P)/dw, (H-2P)/dh)`
4. 居中偏移： `offset_x = (W - dw*scale) / 2` ， `offset_y = (H - dh*scale) / 2`
5. **Y 轴翻转**（DXF Y 向上，SVG Y 向下）：

```
sx = round((x - dxf_min_x) * scale + offset_x)
sy = round((dxf_max_y - y) * scale + offset_y)
```

2.3 边界条件

输入	行为
`dxf_bbox` 无数据	抛 `ValueError("empty bbox")`
`viewport` 任意维度 ≤ 2*padding	抛 `ValueError("viewport too small")`
`dw` 或 `dh` = 0	抛 `ValueError("degenerate bbox")`

---

3. `outline_extractor.py`

3.1 公共 API

```
def extract(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
) -> Outline: ...
```

3.2 算法（按优先级降级）

策略 **A**（首选）：扫描 `layer in {"0-立面(轮廓)", "轮廓", "OUTLINE", "A-WALL-EXTR"}` 的 `LWPOLYLINE`，找闭合且周长最大的一条 → 用其顶点构造 polygon。

策略 **B**（无标识图层时）：取所有墙体相关图层（`WALL/CURTWALL/外墙/0muqiang/A-SECT-MCUT`）的 LINE/LWPOLYLINE，做凸包 → 输出 rect bbox。

策略 **C**（兜底）：所有实体的 bbox → 输出 rect。

3.3 输出规则

- 顶点数 ≤ 4 且接近矩形（每个角 90°±5°）→ `Outline(type="rect")`
- 否则 → `Outline(type="polygon", points=[...])`，最多保留 32 个顶点（道格拉斯-普克算法简化）

---

4. `structure_detector.py`

4.1 公共 API

```
def detect(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
    blocks: ezdxf.layouts.BlockLayout,   # doc.blocks，用于查 INSERT 块定义
) -> List[Structure]: ...
```

4.2 识别规则表（按 type 顺序执行）

4.2.1 `elevator`（电梯）

匹配条件（满足任一）：

- INSERT 块名含正则 `r"^DT|电梯|ELEV|ELE\d|EL_\d"` （不区分大小写）
- 实体 layer 名含 `"电梯"` 或 `"ELEV"`
- 邻近（≤ 500mm）有 TEXT/MTEXT 内容为 `"DT\d"` 或 `"E\d"`

输出：

- `rect` = 该 INSERT 的 bbox 转 viewport
- `code` = 按 X 坐标从左到右编号 `E1, E2, ...`
- `confidence` ：块名直接匹配 → 1.0；layer 匹配 → 0.8；邻近文字匹配 → 0.6

4.2.2 `stair`（楼梯）

匹配条件：

- INSERT 块名含 `r"^LT|楼梯|STAIR|STR_"`
- 或图层 `"楼梯"` / `"STAIR"` / `"A-FLOR-STRS"` 上有 HATCH（楼梯踏步斜纹标记）

输出 `label="消防楼梯"` （默认）或 `"楼梯"` （无 INSERT 块名包含「消防」/「FIRE」时）。

4.2.3 `restroom`（卫生间）

匹配条件：

- INSERT 块名含 `r"WC|卫生间|TOILET|男卫|女卫"`
- 或邻近 TEXT 为 `"男卫"` / `"女卫"` / `"M-WC"` / `"F-WC"`

`gender` 推断：

- 文字含「男」/「M」→ `"M"`
- 文字含「女」/「F」→ `"F"`
- 否则 → `"unknown"` （必须由人工确认）

4.2.4 `equipment`（设备间）

匹配条件：

- 邻近 TEXT 包含「设备」/「机房」/「弱电」/「强电」/「水暖」
- 或图层 `"设备"` / `"MEP"` / `"机房"`

4.2.5 `core`（核心筒）

计算策略：在已识别出 elevator + stair + restroom 的前提下，求三者 rect 的最小外接矩形（含 5% 外扩） → 作为 core。

若上述三者数量 < 2，**不输出 core**（ `confidence=0` 也不输出，让人工标注补全）。

输出 `label="核心筒"` 。

4.2.6 `corridor`（走廊）

当前阶段不自动识别，永远返回空 → 由人工在 Admin 标注页新增。

4.3 去重 / 冲突处理

- 同一 INSERT 实体不得被多个 type 命中；命中优先级：`elevator > stair > restroom > equipment`
- 两个相同 type 的 rect IoU > 0.5 → 合并为外接矩形
- `core` 与 `elevator/stair/restroom` 必然空间重叠，不去重（前端绘制时按数组顺序，core 在最底层）

4.4 输出排序

按 `type` 优先级（core > corridor > elevator > stair > restroom > equipment）→ 同类型按 X 升序 → Y 升序。便于人工审核时阅读。

---

5. `column_detector.py`

5.1 公共 API

```
def detect(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
) -> List[Column]: ...
```

5.2 算法

1. 收集图层 `"柱"` / `"COLU"` / `"A-COLS"` / `"柱墙填充"` 上的 SOLID/HATCH/LWPOLYLINE
2. 对每个实体计算 bbox：
   - 长宽 < 1500mm 且长宽比 ≤ 2.5 → 视为柱
   - 否则跳过（可能是其他装饰填充）
1. 取中心点坐标，转 viewport
2. 网格化去重：以 100mm 为格子合并临近中心点（同一柱可能由多个实体绘制）

5.3 输出限制

- 单层柱数量上限 100；超出按面积大小取前 100，剩余记 warning
- 输出按 `(y, x)` 升序

---

6. `window_detector.py`

6.1 公共 API

```
def detect(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
    outline: Outline,
```

```
) -> List[Window]: ...
```

6.2 算法

1. 找图层 `"WINDOW"` / `"窗"` / `"A-GLAZ"` 的 LINE/LWPOLYLINE → 优先策略
2. 若无窗图层：扫描 outline 矩形四条边，找位于边上的"双线断口"（外墙双线之间的开口） → 兜底启发式
3. 对每个候选窗：
   - 判断最近邻 outline 边 → 设 `side` （N/S/E/W）
   - `offset` = 窗中点沿该边的偏移量（像素）
   - `width` = 窗宽度（像素），最小 8px

6.3 输出限制

- 单层窗洞总数上限 100
- 超出时按等距网格聚类（保留分布特征，丢失精度）

---

7. 与主流程的集成

7.1 修改 `split_dxf_by_floor.py`

在 `render_region_to_svg()` 函数内、写 `_inject_hotzone_spec` 之后，新增：

```
from floor_map import (
    coordinate_mapper, outline_extractor,
    structure_detector, column_detector, window_detector,
)

mapper     = coordinate_mapper.build(actual_bb, viewport=(json_skeleton["viewport"]["width"],
                                                          json_skeleton["viewport"]["height"]))
outline    = outline_extractor.extract(in_region, mapper)
structures = structure_detector.detect(in_region, mapper, doc.blocks)
columns    = column_detector.detect(in_region, mapper)
windows    = window_detector.detect(in_region, mapper, outline)

json_skeleton["schema_version"] = "2.0"
json_skeleton["render_mode"]    = "vector"
json_skeleton["outline"]        = asdict(outline)
json_skeleton["structures"]     = [asdict(s) for s in structures] + [asdict(c) for c in
columns]
json_skeleton["windows"]        = [asdict(w) for w in windows]
json_skeleton["north"]          = None  # 始终待人工补充
```

7.2 输出双文件

- `A座_F<N>.json` — 主文件，仅含 `source="manual"` 项 + 自动识别但 `confidence ≥ 0.9` 的项
- `A座_F<N>.candidates.json` — 候选项全集（所有 auto 项含 confidence），供 Admin 标注页对比

首次抽取时两份内容相同；后续人工编辑只更新主文件。

8. 单元测试

测试目录：`backend/test/floor_map_extractor_test.py`（虽然在 backend/，但调用 Python 脚本，可用 `Process.run`）。

每个 detector 至少覆盖：

用例	期望
标准 A 座 F11 fixture	识别出 4 部电梯 + 2 部楼梯 + 4 个卫生间
空 DXF 文件	全部返回 `[]`，不抛错
仅含外轮廓	`outline.type == "rect"`
多边形外轮廓	顶点数 ≤ 32
柱位 200 个	输出截断为 100，warning 计数 = 100
缺失图层信息（仅几何）	启发式仍能识别 ≥ 50%

Fixture 文件放：`backend/test/fixtures/floor_map/<name>.dxf`（小型最小可用 DXF，每个 ≤ 100KB）。

9. 性能要求

- 单楼层抽取耗时 ≤ 3 秒（24 层全栋 ≤ 90 秒）
- 内存峰值 ≤ 500MB
- 不并发执行楼层抽取（ezdxf 非线程安全）

10. 日志与可观测性

每个 detector 必须输出统一格式日志：

```
[floor_map.<detector>] floor=F<N> auto=<n> manual=0 conf_avg=<float> warn=<n>
```

汇总到 stderr，集成 scripts/build_floors.sh 末尾打印。

Floor Map Extractor 详细规范

> 版本：v1.0
>
> 关联：FLOOR_MAP_HYBRID_RENDERING_PLAN.md §4
>
> 目标读者：实现 `scripts/floor_map/*.py` 的开发者（人或 Copilot）

本文档规定每个抽取模块的输入 / 输出 / 算法 / 边界条件 / 单元测试要求。所有模块都是纯函数，不修改 DXF。

---

0. 公共类型定义

```
# scripts/floor_map/types.py
from dataclasses import dataclass
from typing import Literal, Optional, List, Tuple

Point = Tuple[float, float]   # SVG viewport 坐标（已映射）

@dataclass(frozen=True)
class Rect:
    x: float; y: float; w: float; h: float

@dataclass(frozen=True)
class Outline:
    type: Literal["rect", "polygon"]
    rect: Optional[Rect] = None
    points: Optional[List[Point]] = None

StructureType = Literal[
    "core", "elevator", "stair", "restroom",
    "equipment", "corridor", "column",
]

@dataclass(frozen=True)
class Structure:
    type: StructureType
    rect: Optional[Rect] = None       # column 用 point 而非 rect
    point: Optional[Point] = None
    label: Optional[str] = None
    code: Optional[str] = None        # 电梯 E1/E2
    gender: Optional[Literal["M", "F", "unknown"]] = None
    source: Literal["auto", "manual"] = "auto"

@dataclass(frozen=True)
class Window:
    side: Literal["N", "S", "E", "W"]
    x: Optional[float] = None         # N/S 用 x
    y: Optional[float] = None         # E/W 用 y
    width: float = 30.0
```

---

1. coordinate_mapper

职责：把 DXF mm 浮点坐标映射到 SVG viewport 整数坐标。

1.1 接口

```
class CoordinateMapper:
    def __init__(self, dxf_bbox: BoundingBox, viewport: Tuple[int, int]):
        ...
    def to_viewport(self, x: float, y: float) -> Tuple[int, int]:
        ...
    def to_viewport_rect(self, dxf_min: Point, dxf_max: Point) -> Rect:
        ...
```

1.2 算法

1. 计算缩放比 `s = min(viewport_w / dxf_w, viewport_h / dxf_h)` ，**保持长宽比**（不允许各向异性缩放）
2. 居中偏移 `tx = (viewport_w - dxf_w * s) / 2` 、 `ty = (viewport_h - dxf_h * s) / 2`
3. **Y 翻转**：DXF 上正、SVG 下正 → `sy = viewport_h - (y - dxf_min_y) * s - ty`
4. 量化到 0.1 像素（避免浮点抖动）： `round(v * 10) / 10`

1.3 边界

- DXF bbox 退化（ `w == 0` 或 `h == 0` ）→ 抛 `ValueError`
- viewport 必须为 `(int, int)` 且 ≥ 100

1.4 单元测试

Case	期望
DXF bbox `(0,0)–(8410,5940)` → viewport `(1200,800)`	比例 0.1426，整图居中，Y 翻转
单点 `(0,0)` （DXF 左下角）	viewport `(0, 800)` 附近（含居中偏移）
单点 `(8410,5940)` （DXF 右上角）	viewport `(1200, 0)` 附近

---

2. outline_extractor

职责：识别楼层外轮廓矩形 / 多边形。

2.1 接口

```
def extract(entities: List[DXFEntity], mapper: CoordinateMapper) -> Outline
```

2.2 算法（按优先级）

1. **优先**：layer 在 `OUTLINE_LAYERS = {"0-立面(轮廓)", "OUTLINE", "A-SECT-MCUT"}` 内，且为闭合 LWPOLYLINE / POLYLINE，取最大面积者
2. **次选**：扫描所有 LINE / LWPOLYLINE，构建无向图，找最大连通区域的凸包

3. 降级：所有实体的 bbox 直接作 outline `type="rect"`

2.3 输出规则

- 若 polygon 顶点 ≤ 8 → `type="polygon"`，导出顶点
- 否则 → 求外接矩形（`bbox`），`type="rect"`

2.4 边界

- 实体为空 → 抛 `ValueError("empty floor region")`

2.5 单元测试

固定一份 fixture DXF（A 座 F11 子集），断言输出 `Rect.w` 在期望值 ±2% 内。

---

3. structure_detector

职责：识别 core / elevator / stair / restroom / equipment / corridor。

3.1 接口

```
def detect(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
    blocks: BlocksSection,
) -> List[Structure]
```

3.2 识别策略表

类型	识别依据（按优先级）	备注
`elevator`	INSERT 块名匹配 `r"(DT	电梯
`stair`	INSERT 块名匹配 `r"(LT	楼梯
`restroom`	INSERT 块名匹配 `r"(WC	卫生间
`equipment`	layer 含 "设备" 或 "MEP"	label = layer 名
`corridor`	layer = "PUB" 的非闭合长条 polygon（长宽比 ≥ 4:1）	rect = bbox
`core`	包围所有 elevator + stair 的最小外接矩形 + 50 像素 padding	必须放在最后计算

3.3 关键约束

- 不允许直接处理 `column`（拆给 column_detector）
- 不允许修改 DXF（任何 `e.dxf.X = ...` 都禁止）
- 块名匹配大小写不敏感，且 `re.search` 而非 `re.match`
- 同类去重：rect IoU ≥ 0.7 视为重复，保留 bbox 较大者

3.4 输出顺序

```
[core, elevator(E1), elevator(E2), ..., stair, restroom(M), restroom(F), equipment, corridor]
```

固定顺序便于 diff、便于前端渲染层叠。

3.5 单元测试

场景	期望
F11 fixture	core=1, elevator=4, stair=2, restroom={M,F}
无电梯楼层（F1 大堂）	elevator=0, core=0（无 elevator/stair 时不构造 core）
块名 `D-T-01` （带分隔符）	命中 elevator 正则

---

4. column_detector

职责：识别柱位中心点。

4.1 接口

```
def detect(entities: List[DXFEntity], mapper: CoordinateMapper) -> List[Structure]
```

4.2 算法

1. 收集 layer 含 "柱" 或 "COLUMN" 的所有 HATCH / SOLID 实体
2. 取每个实体的 bbox 中心 → mapper.to_viewport
3. 聚合：欧氏距离 < 8 像素的合并为一个柱（去重多重描边）
4. 按 (y, x) 排序，输出 `Structure(type="column", point=(x, y))`

4.3 边界

- 单个柱的 bbox 边长 > 30 像素 → 视为非柱（可能是墙体填充），跳过
- 柱总数 > 200 → 抛日志警告，但不截断

4.4 单元测试

场景	期望
F11 fixture	柱数量 ∈ [10, 30]
重复实体（双圈柱）	仅输出 1 个

---

5. window_detector

职责：识别外墙窗洞位置。

5.1 接口

```
def detect(
    entities: List[DXFEntity],
    mapper: CoordinateMapper,
    outline: Outline,
) -> List[Window]
```

5.2 算法

1. 优先：layer 含 "WINDOW" / "窗" 的 LWPOLYLINE，取 bbox 中心投影到 outline 边
2. 次选：扫描外墙 LINE，找"断点"（两段连续墙线之间的间隙长度在 [800mm, 4000mm]） → 视为窗洞
3. 投影到 N/S/E/W：
   - bbox 中心距 outline 顶/底/左/右边距离最小者为该 side
   - N/S 输出 `x`，E/W 输出 `y`
1. 限流：每 side 最多输出 50 个；超过时按等距聚类（ `k=50` ）

5.3 输出顺序

按 `(side顺序: N,S,E,W, 然后 x/y 升序)` 。

5.4 单元测试

场景	期望
F11 fixture	N + S 总数 $\in [10, 30]$
无 WINDOW 图层	走断点降级路径，至少识别 50%

---

6. 集成入口

`scripts/split_dxf_by_floor.py` 末尾追加：

```
from floor_map import (
    coordinate_mapper, outline_extractor,
    structure_detector, column_detector, window_detector,
)

mapper = coordinate_mapper.CoordinateMapper(actual_bb, viewport=(1200, 800))
outline    = outline_extractor.extract(in_region, mapper)
structures = structure_detector.detect(in_region, mapper, doc.blocks)
structures += column_detector.detect(in_region, mapper)
windows    = window_detector.detect(in_region, mapper, outline)
```

```
# 写入 floor_map_v2 骨架
floor_map["schema_version"] = "2.0"
floor_map["render_mode"]    = "vector"            # 默认仍走矢量
floor_map["outline"]        = asdict(outline)
floor_map["structures"]     = [asdict(s) for s in structures]
floor_map["windows"]        = [asdict(w) for w in windows]
floor_map["north"]          = None                # 永远人工标注
```

禁止在集成入口里写任何识别逻辑——保持入口只做编排。

---

7. 测试基线

`backend/test/floor_map/` 或 `scripts/test_floor_map.py`：

```
test_coordinate_mapper.py
test_outline_extractor.py
test_structure_detector.py
test_column_detector.py
test_window_detector.py
fixtures/
  A_F11_minimal.dxf       # 仅含 F11 区域、剥离了无关楼层
  expected_F11.json       # 期望输出
```

每次修改算法后必须运行 `pytest scripts/test_floor_map.py`，并对比 `expected_F11.json` 做差异回归。

---

8. 反模式（不要这么做）

🚫 不要 在 detector 内部递归遍历 `doc.blocks`（除 structure_detector 显式需要 INSERT 块名外）

🚫 不要 在 detector 内部 print 日志（用 logging.getLogger(**name**).debug）

🚫 不要 把楼层号、楼栋号当 detector 参数（detector 只看实体集合）

🚫 不要 在 detector 中混合"渲染"和"抽取"（detector 不输出 SVG，只输出结构化数据）

🚫 不要 跨楼层共享缓存（每层调用都是独立上下文）